Répétition et principe DRY

Les boucles les boucles les boucles les boucles les boucles les boucles les boucles les boucles...

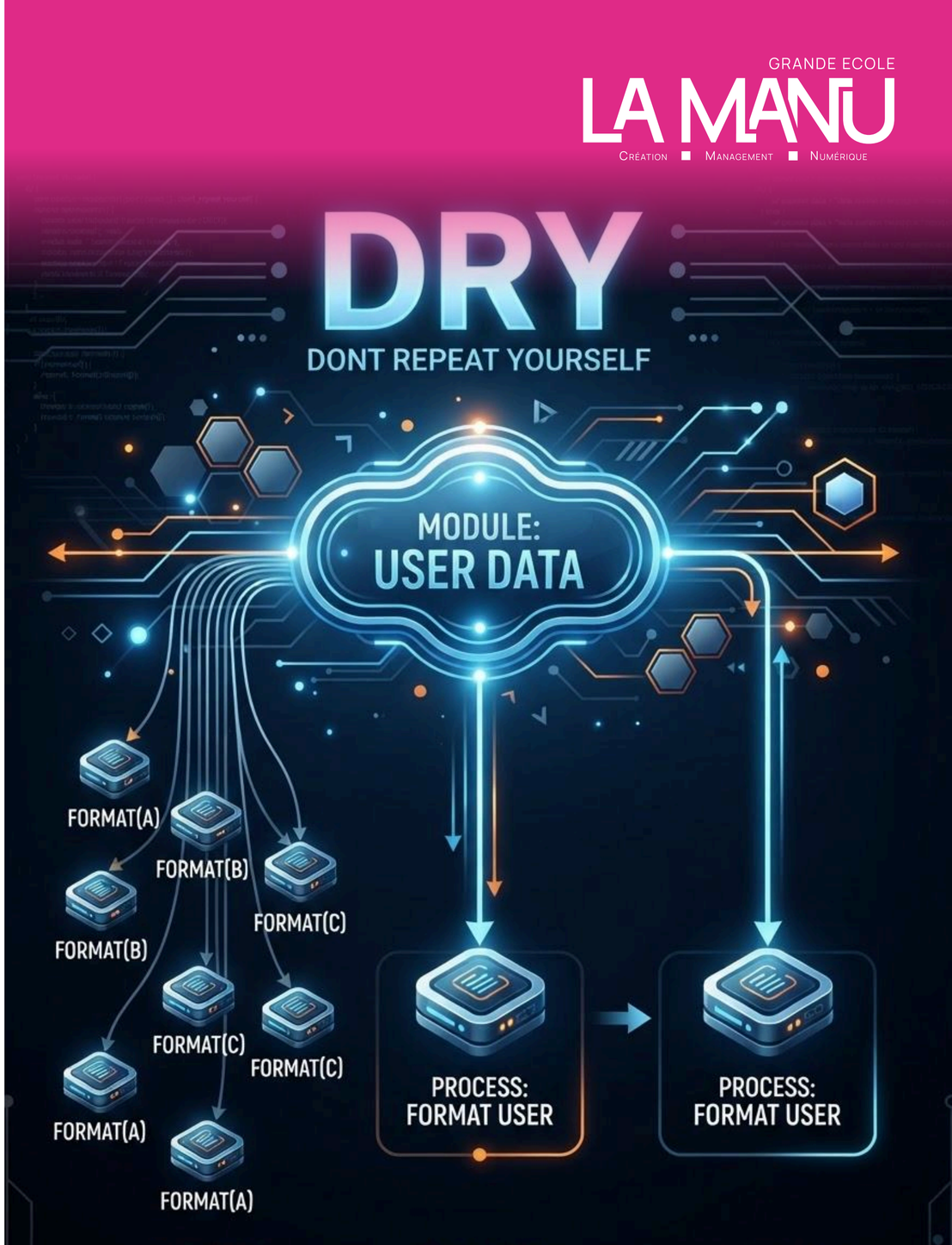


Le problème de la duplication

Imaginez que vous deviez afficher **100** fois "Bienvenue sur notre site" ou afficher les **50** produits d'une base de données e-commerce.

Si vous deviez **écrire 50 fois** l'instruction d'affichage, le code deviendrait rapidement illisible, lourd et impossible à maintenir.

C'est la naissance du principe **DRY** (**Don’t Repeat Yourself**) en programmation : **Ne jamais dupliquer du code**



Les boucles

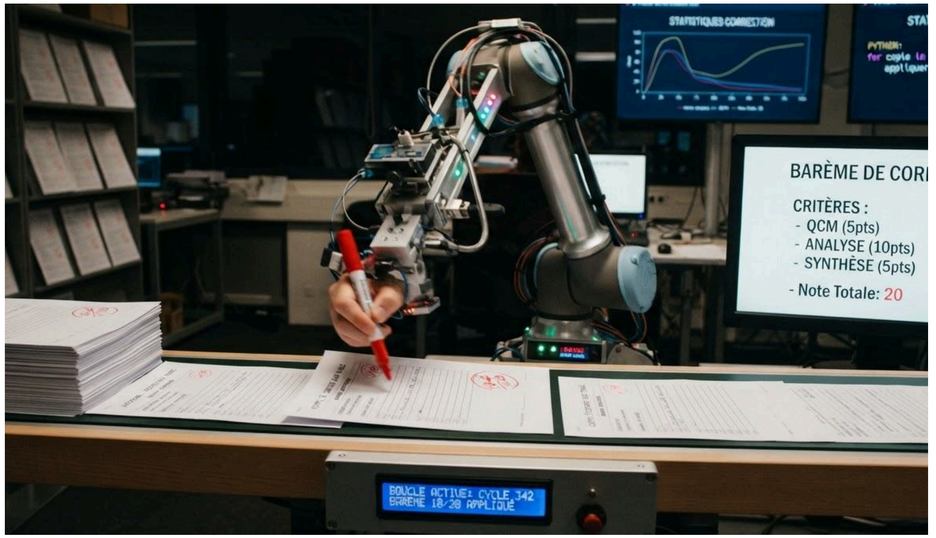
GRANDE ECOLE
LA MANU
CRÉATION ■ MANAGEMENT ■ NUMÉRIQUE

Quelle est la différence entre :



Un professeur qui corrige 30 copies





Un robot qui corrige 30 copies

TANT QUE vs POUR

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Une **boucle** permet d'exécuter **un bloc d'instructions** de manière **répétée** tant qu'une règle est respectée ou pour chaque élément d'une collection.

En algorithmique, on distingue **deux grands types** de boucles selon que l'on connaît ou non le nombre de répétitions à l'avance.

La boucle conditionnelle : TANT QUE (While)

On répète un bloc d'instructions tant qu'une condition reste vraie.

On ne sait pas à l'avance combien de fois la boucle va tourner.

La boucle bornée : POUR (For)

On répète un bloc d'instructions un nombre de fois précis et connu à l'avance, ou pour chaque élément d'une collection.

La boucle TANT QUE (While)

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Anatomie et cycle de vie :

1. **L'initialisation** : La variable testée doit exister et avoir une valeur avant d'entrer dans la boucle.
2. **Le test d'entrée** : A chaque tour, l'ordinateur évalue la condition. SI c'est VRAI, on entre, SI c'est FAUX, on saute directement après le FIN TANT QUE.
3. **Le corps de la boucle** : Les instructions exécutées en boucle
4. **La mise à jour (Cruciale)** : Au moins une instruction à l'intérieur du bloc doit modifier la variable de la condition pour qu'elle devienne fausse un jour.

```
1  essais <- 0
2  code_valide <- FAUX
3
4  TANT QUE (code_valide == FAUX) ET (essais < 3) FAIRE
5      Afficher("Entrez le code secret : ")
6      Saisir(saisie_utilisateur)
7
8      SI saisie_utilisateur == "4321" ALORS
9          code_valide <- VRAI
10     SINON
11         essais <- essais + 1
12         Afficher("Erreur, il vous reste ", 3 - essais, " essais.")
13     FIN SI
14 FIN TANT QUE
15
16 SI code_valide ALORS
17     Afficher("Accès autorisé.")
18 SINON
19     Afficher("Compte bloqué.")
20 FIN SI
```

La boucle POUR (For)

GRANDE ECOLE
LA MANU
CRÉATION ■ MANAGEMENT ■ NUMÉRIQUE

La boucle bornée s'utilise lorsque le nombre d'itérations est connu avant même de démarrer la boucle.

Elle automatise la gestion d'un compteur.

```
// Anatomie et syntaxe complète :
POUR variable_compteur ALLANT DE debut A fin [PAS DE pas] FAIRE
    // Instructions répétées
FIN POUR

// Exemple
// Afficher une table de compte à rebours pour un lancement
POUR t ALLANT DE 5 A 1 PAS DE -1 FAIRE
    Afficher("T moins ", t)
FIN POUR
Afficher("Décollage !")
```

Ce qui se passe sous le capot

Un POUR est, sur le plan logique, un raccourci syntaxique élégant d'un TANT QUE.

```
i <- debut
TANT QUE i <= fin FAIRE
    // Instructions
    i <- i + pas
FIN TANT QUE
```

Répétition et principe DRY

Les listes / collections



Le problème des variables simples

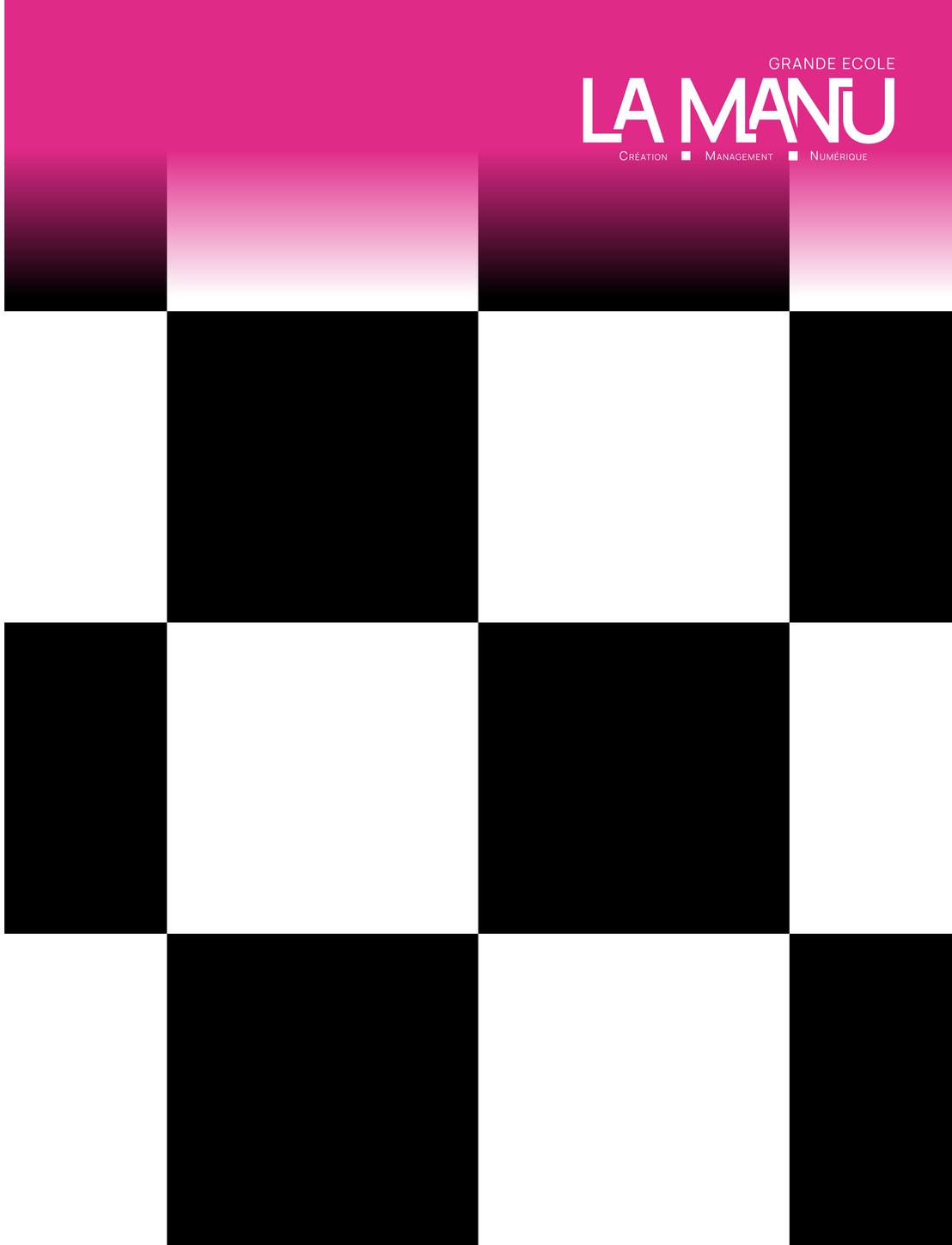
GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Pourquoi stocker en collection ?

Jusqu'à présent, chaque variable ne pouvait stocker qu'une seule valeur à la fois (ex: note = 14).

Si un enseignant doit gérer les notes de 30 étudiants, créer 30 variables distinctes (note1, note2, note3...) rendrait l'algorithme ingérable et l'automatisation par boucle impossible.

La **collection** (souvent appelée **tableau** ou **array** ou **liste**) permet de regrouper un ensemble de valeurs de **même type** sous un nom unique, **ordonnées** les unes à la suite des autres dans la mémoire de l'ordinateur.



L'anatomie d'un tableau et la règle de l'indexation à 0

GRANDE ECOLE LA MANU
Création ■ Management ■ Numérique

Un tableau se compose d'une suite de cases mémoires. Chaque case possède un numéro d'ordre appelé indice (ou index).

```
1  // Pour le déclarer, on note les valeur séparer par des
   virgules entre 2 crochets :
2  notes <- [12, 13, 17, 8]
3  // Modifier la valeur du 2e élément (index 1)
4  notes[1] <- 13
5  // Stocker ou afficher la valeur du premier élément
6  Afficher("La première note est : ", notes[0])
```

La règle d'or (Piège classique) : En informatique, la très grande majorité des langages commencent à compter les indices à partir de 0, et non de 1.

Exemple concret : Soit un tableau de 4 produits e-commerce nommé panier.

- panier[0] → Contient "Casque" (1er élément, index 0)
- panier[1] → Contient "Clavier" (2e élément, index 1)
- panier[2] → Contient "Souris" (3e élément, index 2)
- panier[3] → Contient "Écran" (4e élément, index 3)

Le parcours automatique

La véritable puissance des tableaux réside dans leur association avec les boucles (particulièrement la boucle POUR).

Pour parcourir l'intégralité d'un tableau de la première à la dernière case sans dupliquer le code, on pilote l'index à l'aide du compteur de la boucle.

```
1  // Afficher tous les éléments d'un tableau de 5 notes
2  POUR i ALLANT DE 0 A 4 FAIRE
3      Afficher("Note à l'index ", i, " : ", notes[i])
4  FIN POUR
```

Le parcours automatique

La véritable puissance des tableaux réside dans leur association avec les boucles (particulièrement la boucle POUR).

Pour parcourir l'intégralité d'un tableau de la première à la dernière case sans dupliquer le code, on pilote l'index à l'aide du compteur de la boucle.

Le dépassement d'indice (Out of Bounds)

Un tablau de 5 éléments (indices de 0 à 4)
Si **notes[5]**
Le programme va tenter de lire une case mémoire située en dehors des limites allouées.

Conséquence : Une erreur critique de type **IndexOutOfBoundsException**

```
1    // Afficher les 5 notes du tableau de 5 notes
2    POUR i ALLANT DE 0 A 4 FAIRE
3        Afficher("Note à l'index ", i, " : ", notes[i])
4    FIN POUR
```

A VOUS DE JOUER

2ème TD